

Projet Individuel — Sokoban

Objectif

Réaliser une application JavaFX permettant de jouer au jeu **Sokoban** (le gardien d'entrepôt).

Vous devez créer une interface graphique **ergonomique et fonctionnelle** en utilisant les classes métier fournies (le "moteur de jeu") **sans les modifier**.

Vous disposez de **2 jours** (+ le week-end si nécessaire) pour réaliser ce projet.

Consultez le fichier `description-sokoban.pdf` si vous ne connaissez pas les règles du jeu.

Consultez le fichier `GuideUtilisation.pdf` pour comprendre comment utiliser le moteur de jeu.

Le projet `demo_console_moteur.zip` montre comment utiliser le moteur **sans IHM** : à étudier en premier pour bien comprendre l'API avant de coder l'IHM JavaFX.

Contraintes

- Le projet doit être développé en **JavaFX avec FXML** (comme vu en cours).
- Vous devez utiliser les classes fournies dans les packages `ihm.sokoban.model` et `ihm.sokoban.util` **sans les modifier**. Cette contrainte sera vérifiée à la correction.
- Le projet doit être un projet **Maven** (basé sur le modèle HelloJavaFX).
- Le code doit être **lisible, commenté** et **organisé** en packages.
- Vous devez rendre un **JAR exécutable** (généré par Maven) et un **document de synthèse** (PDF, 1-2 pages) présentant votre projet, les options implémentées, les difficultés rencontrées, etc.

Trois paliers de périmètre

Le projet est gradué en trois paliers. **Le palier A est obligatoire pour tous**. Les paliers B et C sont optionnels et permettent de viser une note plus élevée. Vous pouvez choisir librement votre cap.

Palier A — Tutoriel 8×8 (*obligatoire*)

Toute IHM doit pouvoir jouer **les 5 niveaux du mode tutoriel** (`NiveauxTutoriel`), tous au format **8×8**. Le périmètre minimal n'impose rien d'autre :

- Vous pouvez supposer que **la grille fait toujours 8×8**.
- Vous pouvez créer une grille fixe dans le FXML (8 lignes × 8 colonnes).
- Le joueur doit pouvoir **enchaîner les 5 niveaux jusqu'à la victoire finale**.

Palier B — 10 niveaux variés

Votre IHM gère les **10 niveaux** de `NiveauxSokoban` (de 5×5 à 10×9). Cela impose de **construire dynamiquement la grille** en fonction des dimensions retournées par `jeu.getNbLignes()` / `jeu.getNbColonnes()`.

Palier C — Dossier .xsb

Votre IHM permet à l'utilisateur de **choisir un dossier** contenant des fichiers Sokoban au format standard XSB (001.xsb, 002.xsb, ...). L'archive `niveaux_xsb_exemple.zip` contient 12 niveaux d'exemple.

Un loader **clé en main** est fourni : `LoaderNiveauxXSB.chargerDepuisDossier(Path)`. Aucun parsing à coder, vous gérez juste la **UX** (`DirectoryChooser`, gestion d'erreurs).

Fonctionnalités minimales (palier A)

Toutes ces fonctionnalités doivent être implémentées pour valider le palier A :

- 1. **Affichage du plateau** : couleurs ou symboles distincts pour chaque type de case (mur, sol, cible, caisse, joueur). Caisse bien placée visuellement différente.
- 2. **Déplacement du joueur** aux **flèches** du clavier, en respectant les règles (murs, poussée).
- 3. **Poussée des caisses** : poussée si la case derrière est libre, sinon mouvement bloqué (pas de crash).
- 4. **Détection de victoire** : message visible quand toutes les caisses sont sur les cibles.
- 5. **Détection de défaite (état PERDU)** : si le moteur détecte qu'une caisse est coincée dans un coin, l'IHM doit l'indiquer clairement et **proposer Annuler ou Recommencer**. *Le moteur lève une exception si vous tentez `deplacer()` après la fin de partie : votre IHM doit donc filtrer.*
- 6. **Compteur de mouvements** affiché.
- 7. **Recommencer** le niveau (bouton ou touche).
- 8. **Quitter** l'application proprement (avec confirmation).
- 9. **Enchaîner les 5 niveaux du tuto** : un bouton/menu *Niveau suivant* permet de passer au niveau suivant. Quand le **dernier** niveau est terminé, l'IHM affiche un message **différent** (exemple : *"Bravo, dernier niveau réussi !"*) du simple message *"Niveau réussi"*.

Fonctionnalités optionnelles

Les options sont classées en **deux catégories** : **Jeu / Sokoban** (logique du jeu, niveaux, scores) et **JavaFX / IHM** (présentation, interaction, animation). Pour viser les meilleures notes, **équilibrez** vos options entre les deux catégories.

Catégorie 🎮 Jeu / Sokoban

Difficulté	Option
★	Compteur de poussées (en plus des mouvements).
★	Nom du niveau affiché.
★★	Annuler (Undo) — bouton et <code>Ctrl+Z</code> .
★★	Navigation entre niveaux — boutons précédent / suivant.
★★	Sélection du niveau — menu ou liste déroulante.
★★	Passer au palier B (10 niveaux variés, grille dynamique).
★★★	Progression auto — passer automatiquement au niveau suivant après victoire.

Difficulté	Option
★★★	Meilleur score par niveau (mouvements / poussées).
★★★	Passer au palier C (chargement de dossier <code>.xsb</code>).
★★★★	Détection de blocage avancée (au-delà du corner-trap déjà géré par le moteur).
★★★★	Sauvegarde des scores (sérialisation / fichier).
★★★★	Éditeur de niveaux intégré.

Catégorie 🧠 JavaFX / IHM

Difficulté	Option
★	Personnalisation CSS : couleurs, style des cases.
★	Menu "À propos" avec votre nom et description.
★★	Affichage avec emojis ou symboles Unicode (📦, 🎯, 📦, 🧑).
★★	Raccourcis clavier (<code>R, Z, N, P, Ctrl+Z...</code>).
★★	MenuBar structurée (Jeu / Édition / Aide).
★★★	Affichage avec images / sprites (<code>ImageView</code>).
★★★	Compteurs réactifs avec <code>IntegerProperty</code> + <code>binding</code> .
★★★	Indicateur de progression : barre ou label " <i>Caisses placées : 2/4</i> ".
★★★	Animation de déplacement avec <code>TranslateTransition</code> ou <code>Timeline</code> .
★★★★	Sons : déplacement, poussée, victoire.
★★★★	Thèmes graphiques : plusieurs jeux de couleurs/sprites, changement dynamique.
★★★★	Redimensionnement dynamique : la taille des cases s'adapte à la fenêtre.

Toute option non listée mais implémentée proprement et présentée dans le document de synthèse peut être valorisée.

Évaluation

Le barème est **ouvert** : la note est calculée en additionnant les points obtenus, avec **un plafond à 20/20**.

⚠️ Être noté sur 14 ne veut **pas** dire avoir 14. Cela dépend de la **qualité** de ce qui est codé. ⚠️ Ne **pas** faire d'option n'est pas une faute : un palier A propre peut tout à fait viser 12–14/20.

Pénalités

Les pénalités sont majoritairement liées au non-respect des consignes, il est donc très simple de les éviter en lisant bien le sujet et en suivant les conseils.

- pas de document de synthèse : **-2 points**
- pas de code source : **-18 points**
- classes du moteur non utilisées ou modifiées : **-15 points**
- codage sans aucun FXML : **-10 points**
- pas de JAR exécutable : **-4 points**
- présence de code mort et/ou de fichiers inutiles : **-4 points**
- levée d'exceptions non gérées (crash) : **-4 points**
- affichage console (ex: `System.out.println(jeu)`) laissé dans la version finale : **-2 points**
- rendu en retard : **-2 points/jour**

Bloc de base

Critère	Détail	Points max
Palier A — niveau 8×8 jouable	Affichage correct, déplacement, poussée, victoire, recommencer, quitter, gestion PERDU	/12
Palier A — enchaînement des 5 niveaux	Niveau suivant fonctionnel + message " <i>dernier niveau réussi</i> " différencié	+2 (→/14)

Un projet **palier A complet** est donc noté **sur 14**. C'est la base sur laquelle s'ajoutent les options.

Repartition des points

Critère	Points
Application minimale fonctionnelle	6
Enchaînement des niveaux	2
Clarté et simplicité du code, commentaires	1
Ergonomie de l'interface (utilisation claire, simple et fluide)	2
Design soigné (visuel, lisibilité)	2
Document de synthèse présent et clair	1
Options implémentées (en fonction de leur pertinence)	jusqu'à 6
Bonus (fonctionnalités originales bien pensées)	jusqu'à 2

Bonus de **1 point** si le projet est déposé avant **18h45 vendredi 22 mai**

Note maximale : 22 (Moyenne ramenée à 20 avec QCM)

Bloc options

Bloc	Points max
Options 🎮 Jeu / Sokoban implémentées	+4

Bloc	Points max
Options 🧩 JavaFX / IHM implémentées	+4
Options libres / créativité / polish exceptionnel	+2

La grille de notation laisse donc de la marge à tout le monde. **Soignez ce que vous codez** plutôt que de tout commencer.

Livrables

À la fin du projet, vous devez rendre :

1. **Le projet complet** (nettoyé avec `mvn clean`) sous forme d'**archive ZIP** (avec tous vos fichiers sources, ressources, FXML, etc.). Le projet doit être importable dans un IDE (ex: VScode) et compilable avec Maven.
2. **Le JAR exécutable** (`*-shaded.jar`) généré par Maven.
3. **Un document de synthèse** (PDF, 1-2 pages) contenant :
 - **Palier visé** (A / B / C)
 - **liste des options** implémentées (🎮 et 🧐).
 - un présentation rapide de l'interface
 - un paragraphe « Éléments remarquables » listant vos options personnelles

Pièges à éviter (à lire avant de commencer)

1. **Focus clavier** : si vous utilisez `scene.setOnKeyPressed(...)`, dès qu'on clique un `Button` avec la souris, les flèches arrêtent de fonctionner. Voir le mini projet `gestion_clavier/` pour la bonne approche (`addEventFilter`).
2. **Le moteur est testé**, il fonctionne. Si votre jeu bugge, le bug est dans **votre IHM**, pas dans le moteur. Avant de demander de l'aide, lancez `demo_console_moteur` pour confirmer.
3. **Ne modifiez pas le moteur** (packages `model` et `util`). Cela sera vérifié à la correction.
4. Le moteur **lève une `SokobanException`** si vous appelez `deplacer()` après la fin de partie. **Filtrez** dans votre contrôleur (`jeu.peutJouer()`) avant d'appeler `deplacer()`.
5. **Pas de logique métier dans vos contrôleurs** : utilisez `JeuSokoban`, ne réimplémentez pas `Sokoban` à côté.

Conseils

- **Commencez par lire la documentation du projet.**
- **Faites le palier A en premier**, propre, avant d'envisager B ou C.
- **Testez souvent** : lancez votre application régulièrement.
- `mvn clean` avant de rendre votre projet.